Enhancing the Microservices Architecture for the Internet of Things

Eyhab Al-Masri
School of Engineering and Technology
University of Washington
Tacoma, USA
ealmasri@uw.edu

***Abstract*—Collecting data from smart Internet of Things (IoT) devices is becoming an increasingly essential part of many of the existing industrial applications. The importance of this data collection relies on the fact that it can uncover valuable insights and enable smarter, faster decision making. This enables organizations to quickly adapt to changes in the workflows, reduce downtime and expand the production capacity and enhance the overall operating efficiency. The problem, however, is that many of these Industrial IoT (IIoT) applications can considerably be influenced by the composition of RESTFul APIs and the microservices architecture they integrate. In addition, IIoT applications do not take into consideration the dynamism of service-based environments and requirements. To overcome these challenges, it is essential to consider the Quality of Service (QoS) characteristics of RESTful APIs and microservices particularly that these properties may fluctuate during their lifecycle. In this paper, we introduce a quality-aware microservices' architecture that continuously monitors the behavior of these services in delivering the required functionality. This paper presents experimental validation results and analysis of the presented ideas.**

Keywords—IoT devices, IoT discovery, fog computing, IoT gateways, microservices, REST, RESTful, APIs

I. INTRODUCTION

In typical Industrial Internet of Things (IIoT) applications, it is common that information is collected by physical objects (or devices) and then sent to an IIoT application via communication networks. Given the proliferation of IoT devices and the growing number of IoT cloud-centric solutions, the ability to handle a large magnitude of device types is no longer generic in terms of device discovery in IoT applications such as industrial manufacturing, smart cities, among others. A key challenge into harnessing the data generated by these devices is that many of these devices have limited hardware capabilities that constraint them to connect to the Internet or stream data directly to the cloud.

Furthermore, there are major challenges associated with discovering devices particularly for large IIoT systems that require thousands of devices to cooperate and share data. Furthermore, enabling the interoperability among these devices with respect to the existence of a wide variety of connectivity sensor protocols (e.g. Z-Wave, ZigBee, BLE, Wi-Fi, etc.) is another challenge. This makes the seamless integration of devices and the ability for them to intercommunicate or share data is another major challenge.

IoT gateways help bridge this communication gap between devices and the cloud by offering local processing and storage in addition to the autonomously controlling local devices in many ways. However, in order to maintain the scalability of these gateways, for example, it is common to utilize RESTful APIs and microservices architecture to dynamically integrate devices with any other device. Microservices have become fundamental building blocks of many IIoT applications particularly in the integration of IoT cloud-based platforms (e.g. IBM Bluemix, Siemen's MindSphere, among others). These microservices help provide simpler ways for exchange data between devices particularly those that do not speak the same protocol.

Since many of the tasks provided by IoT devices are mainly sensing and actuating (e.g. sending data or receiving instructions), guaranteeing the continuous availability of microservices without downtime becomes a challenge particularly as number of the microservices attached to a particular workflow increases. This can significantly impact the operating efficiency and production capacities. To illustrate this, consider, for example, a company that operates multiple facilities for food processing. The company's factory utilizes both wired and serial control networks. Since both of these networks are not compatible in terms of the protocols they use, the company installs multiple IoT gateways to help maintain information processing and real-time dashboard for its factory. Multiple microservices are used to enable the serial control network devices to communicate with the wired network devices via a data distribution service (DDS). Consider, for example, that one of the microservices associated with one of the PLC machines malfunctions or is sending incorrect readings or instructions. This can in turn affect, for example, troubleshooting and configuration, remote monitoring or programming, costly downtime, among others. Hence, there is a need for a mechanism that can detect the operational continuity of these microservices using well-defined criteria such as considering their Quality of Service (QoS) attributes.

As microservices integrated into IIoT applications proliferate, the need to establish a universal repository of microservices through a trusted broker that contains a catalog of microservices becomes inevitable so that devices can easily discover them. Furthermore, it is necessary to define appropriate criteria to determine the "best" microservice amongst all possible discovered microservices. In addition, analyzing the quality of individual microservices is imperative

978-1-5386-5035-6/18/$31.00 ©2018 IEEE

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:43:35 UTC from IEEE Xplore. Restrictions apply.

particularly with the increasing demand for providing an acceptable quality over microservices.

To address the above issues, this paper introduces a mechanism that focuses on the utilization of QoS attributes to monitor the behavior of microservices in IIoT applications and thus offering QoS-aware IIoT microservices architecture. Our solution has been tested and results demonstrate the effectiveness of incorporating QoS properties for microservices as part of IoT application integration. Incorporating QoS properties when integrating microservices in IoT applications provides adequate information to service requestors about service guarantees and gives them some confidence as to the quality of microservices they are about to integrate.

The rest of this paper is organized as follows. Section II discusses the related work. Our QoS-aware mechanism is presented in Section III. Experimental results are discussed in Section IV and finally the conclusion and future work are discussed in Section V.

II. Related Work

Multiple microservices may share similar functionalities, but possess different non-functional characteristics. When integrating microservices in IoT applications, it is essential to consider both functional and non-functional properties in order to render an effective and reliable integration process. Unfortunately, many of the current research work has focused on QoS features of the Data Distribution Service (DDS) and does not include QoS metrics for the delivery of requested functionality of RESTful APIs or microservices. In the following paragraphs, we will introduce notable research efforts in the support of the DDS QoS support and QoS management for web services in general.

A. Data Distribution Service (DDS)

Data Distribution Service (DDS) is a data-centric, publish-subscribe distribution middleware protocol and API standard developed by the Object Management Group (OMG) [1]. Unlike the Message Queuing Telemetry Transport (MQTT) and Advanced Message Queuing Protocol (AMQP) protocols, DDS uses a broker-less publish-subscribe architecture. DDS also provides QoS-controlled data-sharing where applications communicate by publishing and subscribing to topics identified by their topic name. DDS sends multicast message to update many remote applications at once and allows data to be shared with flexible QoS specifications such as reliability, system health, security, among others [1]. In dealing with end-to-end QoS of the applications, Real-Time Innovations Inc. (RTI) proposed built-in QoS profiles [2] in the DDS implementation.

In [3], the authors proposed a component platform for developing a framework for fault tolerance and safe inter-component communication related to QoS aspects. In [4], the authors proposed a model-driven approach for enabling the safe, automatic and transparent adaptation of QoS attributed in DDS-based middleware. Although supporting QoS via DDS does not provide reliable mechanism for determining what data it needs to store and controls the sharing of data, it does not provide any capability to extend this QoS support to microservices or RESTful APIs to is nature being data centric and not service-oriented.

Furthermore, some DDS vendors developed modeling tools such as OpenSplice Modeler for creating QoS configurations [5]. These tools provide a graphical user interface (GUI) to represent DDS entities, specify QoS policies and generate implementation artifacts. In [6], the authors present an adaptive Quality Modeling Language (AQML) for modeling, simulating and generating QoS adaptation templates. In [7], the authors proposed a distributed QoS modeling language (DQML) that helps in setting and generating valid QoS configurations in DDS. However, this approach is propitiatory to a specific vendor technology and does not provide any adaptive mechanism to dynamically adjust resources.

In [7], the authors present a middleware that is based on DDS for wireless sensor networks where event routing is used to satisfy QoS characteristics based on existing resources and overall performance. Other researchers focused on building a middleware mechanism for controlling CPU utilization in distributed real-time and embedded systems [8]. Unfortunately, current solutions either focused on an end-to-end quality for data centricity which is very broad or managing fluctuations in the workload of applications and resource utilization which is hardware-oriented. Considering the QoS for microservices in an IoT application has been undermined or often ignored.

B. QoS Support for Web Services

In [9], Ran proposed a system for adding QoS information in service registries using a QoS certification framework. In this model, a service provider prepares a set of QoS measurements and communicates this information to the Certifier which then begins checking QoS claims and sends back a notification to the service provider (i.e. failure acknowledgement or Certification ID). In a similar effort, the authors in [10] proposed QoS support for service-oriented middleware (SoM) where they developed a middleware monitors QoS metrics for services automatically, and four QoS properties were identified: time, cost, reliability, and fidelity.

Another effort presented by the authors in [11] proposed a model for identifying services based on QoS guarantees. In this model, a middleware is introduced to support two groups of non-functional properties: (1) generic quality criteria such as cost, and execution duration; and (2) business related criteria such as compensation and penalty rates. However, in both of these proposed solutions, the authors did not take into consideration QoS properties from various dimensions and only focused on a small set of QoS attributes. In addition, the QoS for authors do not provide an actual implementation of the proposed systems or how QoS metrics are conducted.

The authors in [12] introduced the Web Service Offerings language (WSOL), a language that is used for formal specification of various management statements, constraints, and classes of service for a Web service. In a similar effort, the authors in [13] introduced a model that focuses more on business collaborations and mediating between service providers and clients using service level agreement (SLA). The authors in [14] propose a QoS modeling approach using the Software Workbench for Interactive, Time Critical and Highly self-adaptive cloud applications (SWITCH) IDE. Many of these approaches are either focused on the quality attributes of

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:43:35 UTC from IEEE Xplore. Restrictions apply.

web services or analyze a limited set of non-functional requirements (NFRs).

III. QoS-Aware Microservices

As the number of microservices increases, software engineers and clients will soon face the challenge of differentiating between those microservices that share similar functionalities particularly when operating IoT applications using the fog computing paradigm. Finding relevant microservices will become time consuming and less productive using current techniques. In fact, software developers or engineers may integrate or select microservices that may cause deteriorations to the service workflow which in turn potentially contribute to longer downtime and depreciating the overall performance of IIoT applications.

A. Operating Scenario

To illustrate this, consider Cheese Production using IIoT (CPI) Inc., a cheese production company. CPI has a twenty-year old facility for manufacturing over 15 tons of cheese per day. However, the facility's human machine interface (HMI) application was aging and constantly crashing. During system crashes, staffers and operators would have to stop their work until the system is fully functional. In this case, CPI was risking meeting customer demands and regulatory requirements, increased time to market, deterioration in the overall yield, inability to meet market dynamics and losing valuable data.

In an effort to overcome many of these challenges and improving its overall productivity, CPI has decided to replace its aging production system with a state-of-the-art smart manufacturing plant that integrates sensors and actuators, smart controllers, machines and equipment and IIoT platforms such as Rockwell Automation Integrated Architecture System and Logix control platform, FactoryTalk software suite, a DLR Ethernet network for real-time communication structure. CPI utilizes microservices to speed code reuse, development and deployment. These microservices help addressing assets' connectivity, analytics, security, operations and data distribution.

Due to the fact that IoT devices used in this manufacturing facility speak different protocols, CPI has installed multiple IoT gateways to enable the interconnectivity between devices and the cloud-based IoT platforms (e.g. FactoryTalk Analytics) as shown in Figure 1. CPI has installed sensors into pieces of equipment which continuously monitor their performance. These sensors send their data to a cloud platform which performs advanced analytics and examines historical data as well as purchase data in an effort to determine and predict a possible imminent failure. CPI's IoT system uses a number of in-house and third party microservices to achieve a number of automation tasks including monitoring of machines, alerts, notification, issuing maintenance orders, among many others.

B. Need for a Microservices' Architecture

One of the basic usages of microservices consists of invoking operations via sending and receiving messages. In this scenario, CPI's IoT application that accesses diverse microservices requires advanced features to evaluate or assess

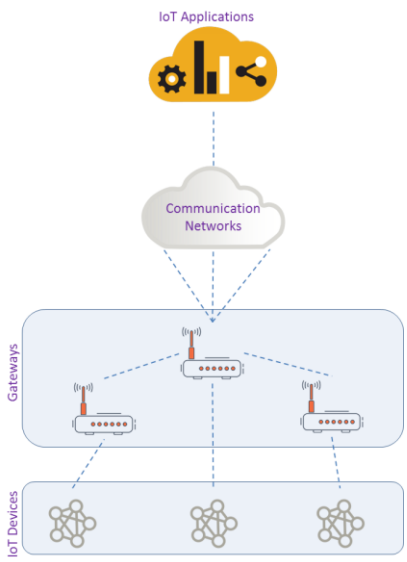

Figure 1. Deployment of IoT Gateways

these microservices prior to any potential interaction. Assessing the behavior of microservices is often neglected by service discovery protocols.

As more microservices become available, a large number of microservices would compete in offering similar functionalities. However, microservices differ in how these functionalities are offered (e.g. input and output data types) and more importantly the conditions to which IoT applications make use of them (e.g. quality of microservices). Unlike web pages which can be ranked based on the popularity of web pages through the linking structure (e.g. web graphs), microservices do not have linking capability. Therefore, an alternative to drive the overall popularity and usefulness of microservices is through the conditions at which these services offer their functionalities, or the Quality of Microservices (mQoS).

C. Assessing Quality of Microservices

To illustrate the importance of incorporating mQoS into the service integration process, consider the following scenario. CPI software developers or engineers are looking for a third party microservice that can provide cloud messaging services (i.e. SMS and text messaging) to be integrated into their IIoT application. CPI's team has identified certain characteristics that this microservice must possess, including:

- fast response time (under 50 ms)
- high availability (99%), and
- low transaction price (US $0.02 or less)

The CPI team performs manual search using existing open-source cloud service platforms such as Mashape [1] and finds that at least 20 third party microservices offer the same functionality. After collecting price details from corresponding service providers, CPI team determines that at least 5 microservices meet their requirements. Which microservice should CPI choose? How much time and effort does CPI team

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:43:35 UTC from IEEE Xplore. Restrictions apply.

have to invest in finding relevant microservices? What distinguishes these microservices from each other?

The choice could be simple if CPI's team assumes that the service providers' QoS claims are trustworthy. In this case, CPI will choose the microservice with fastest response time, highest availability and lowest transaction price. The problem, however, is that the service providers' microservice QoS claims might not always be trustworthy. That is, service providers might significantly influence how these claims are generated and think of ways to improve them. The need for a method to automate the process of measuring mQoS for published microservices becomes inevitable.

Furthermore, CPI requires its IIoT application to continuously monitor critical machine-related sensor thresholds in which if these thresholds are met, the system will dynamically initiate a service or maintenance order. Generating service or maintenance orders is achieved by a microservice. In addition to this, the system can automatically order from third party vendors the necessary replacement parts. Then, upon the receipt of these parts (e.g. delivered to the CPI facility), the system automatically examines proximity and engineers' skills data to route maintenance orders to the appropriate engineer.

For CPI's IoT system to orchestrate and organize the execution of these discrete actions to achieve the desired outcome, a number of RESTful APIs and microservices are required. Integrating these microservices into CPI's IIoT system can achieve all of these tasks with minimal development effort. However, any deterioration or malfunctioning in one or more microservices will likely yield significant downtime, decays in operational efficiency and most importantly an increase in CPI's operating costs.

D. *Containarized IoT Microservices*

CPI's IIoT system utilizes multiple machines, servers, applications, sensors, actuators and communication protocols. Hence, extensive integration between devices, data and applications becomes inevitable. However, integrating IoT components can be costly and time consuming. CPI has decided to utilize containers and microservices. Containers (e.g. Docker) and microservices are applications platforms that emerged for cloud development that are well suited for improving the agility of IoT components integration. Through this model, applications are developed as a suite of small services that execute independently and communicate using lightweight network-based protocols. Therefore, CPI's IIoT system has a number of microservices that are deployed and implemented inside containers. This increases the interoperability of these microservices since there is very little interaction with the underlying operating system. Figure 2 illustrates a segment of CPI's containerized IoT microservices for the operating scenario described earlier.

Although CPI's containerized IoT microservices architecture offers portability, reduced deployment time and minimal interaction with underlying operating systems, there are important challenges that require addressing. More specifically, any deterioration in the performance of one of the microservices can have significant impact on the operating

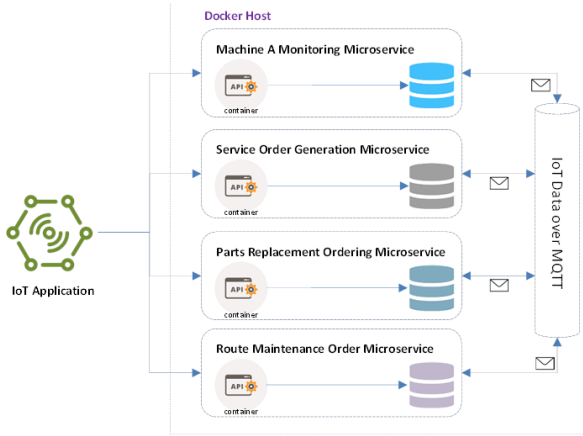

Figure 2: CPI's containerized IoT Microservices

workflow. For example, if the parts replacement ordering microservice malfunctions or takes a significant amount of time to properly execute, this will eventually affect the orchestration of other services required to complete this business process. Although containers enable scalability on demand (i.e. horizontal and vertical), monitoring the behavior of microservices over time is essential.

E. *Addressing Challenges Associated with mQoS*

The fact that microservices operate in a real-time environment (e.g. CPI's manufacturing plant) imposes many constraints such as limited communication bandwidth and the risk of service degradation particularly for low power devices. Consider, for example, that CPI's facility installs a number of drones for the purpose of detecting anomalies or defects in machines or hardware equipment (e.g. dough systems, conveyors, cheese cutters, grinders, cookers, blenders, among others). The drones continuously capture images to be processed by edge devices capable of processing video images locally using machine learning models. The flow of data (e.g. video streams and data generated by sensors or IoT devices) can significantly impact the communication capacity of CPI's local network contributing to having limited or reduced network speed operations. As a result, the transfer of such large data in a continuous manner will eventually affect the network's availability. This can potentially yield to degradation in the overall quality of the microservices in delivering their functionalities.

Assuming that microservices are equal in terms of their behavior in delivering the desired functionality is therefore inadequate. Mechanisms such as Data Distribution Service (DDS) do not take into consideration quality measures for running microservices. Rather, DDS focuses on Quality of Service (QoS) parameters on the delivery of the data without considering the processing time that a microservice consumes to generate or process data. Although DDS's support for QoS is useful with respect to configuring a middleware that can control the end-to-end quality of applications, it does not provide the granularity of determining at the microservices level the behavior at which they are delivering the required functionalities.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:43:35 UTC from IEEE Xplore. Restrictions apply.

IV. MICROSERVICE QOS MANAGEMENT (MQOSM)

QoS characteristics can be expressed in numerical values or quantitative parameters while others could not be defined using a natural metric and mainly reflect the views about a microservice. Quantitative QOS parameters include response time, throughput, availability and reliability. Qualitative parameters include privacy, reputation and cost. When consuming microservices, clients of IoT applications expect a particular QoS level in which with the current architectures are unable to articulate via their requirements precisely. Therefore, a quality-driven mechanism for monitoring microservices becomes a very challenging task. Having an infrastructure where service providers can articulate their QoS requirements properly tailored to their needs is important. However, certain questions when considering QoS for microservices must be addressed which include:

- how would service providers supply QoS for microservices?
- What QoS parameters represent the behavior of microservices?
- How to measure QoS for microservices (mQoS)?
- How to determine the quality level of microservices with respect to the delivery of desired functionality?

Existing software quality standards (such as ISO/IEC 9126, ISO/IEC 12119, and ISO/IEC 14598, OMG's Data Distribution Service – DDS) can be extended to support quality-related issues for microservices and potentially regulate microservice quality requirements, testing and evaluation. Without microservice-quality standards, the task of articulating proper quality-based queries becomes complex [17, 18].

In order to provide a quality of service for microservices (mQoS), it is important to collect QoS information about microservices. To this extent, we developed a framework called microservices QoS Management (mQoSM) that is responsible for determining the overall quality of microservices in an IIoT environment as shown in Figure 3.

A controller module within mQoSM is responsible for coordinating tasks for measuring mQoS. The mQoS Management Capabilities (mQoSMC) is responsible for the discovery, configuration and routing capabilities of microservices. For example, mQoSMC can be used to discovery microservices within an IIoT system and then coordinates with the Scheduler and Controller modules to begin the process of measuring mQoS. The mQoS Operational Capabilities (mQoSOC) is mainly responsible for automating the executing of QoS policies for monitoring mQoS. The mQoSS is responsible for storing the data collected from the other modules including mQoS data such as response time, bandwidth, availability, among others. mQoSM can be used to track microservices local and external to an IIoT system. Furthermore, mQoSM enables clients to selectively choose the microservices to be monitored or can be configured for monitoring to reduce workload and maintain proper load balancing in the microservices execution environment.

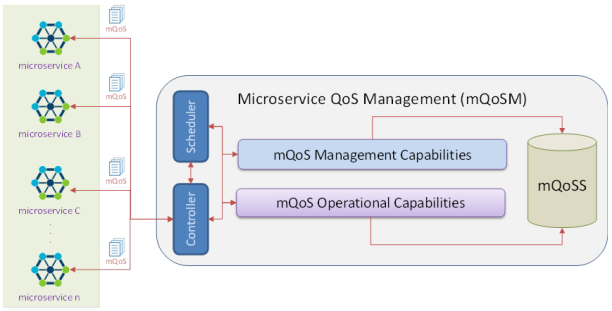

Figure 3. Microservices' Quality of Service Management (mQoSM) Architecture

A. *Microservices Quality of Service (mQoS)*

Quality of Service (QoS) is a broad term and is often associated with network resources and has been widely used in the context of networking and multimedia systems. QoS encompasses the means to predict and manage system resources which are essentially important to the runtime performance of an application and determines the satisfaction of clients of a particular service [19]. QoS is also applicable to other computing resources such as web servers (e.g. service finite number of users over a period of time).

Balancing the load of requests over a web server can potentially improve its overall performance. A differentiating factor between web servers, for example, could be the time to which they are able to service requests (e.g. response time), or the number of concurrent users they could handle (e.g. maximum throughput). By taking these factors into consideration, client applications can efficiently allocate Web server resources based on information such as response time, throughput, among others.

In the case of microservices, they are not monolithic. In fact, a large distributed application can be developed utilizing microservices as the basic computing units. Hence, a large number of competitive implementations for each microservice will exist. The quality of microservices will become a significant differentiator between all of the competing implementations. To that respect, the microservices' quality of service (mQoS) must transcend system-centric quality measures to encapsulate not only implementation details that may influence metrics (e.g. performance) but also deployment and user experience issues.

The mQoS parameters help determine which of the available microservices is the best and meets IoT applications requirements. Due to their significance, we selected the following mQoS parameters taking into consideration the QoS parameters of DDS:

1. response time (rt): the time taken to send a request and receive a response (unit: milliseconds)
2. throughput (tp): the maximum requests that are handled at a given unit in time (units: requests/min)
3. availability (av): a ratio of the time period when a microservice is available (unit: % per 5 day period)

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:43:35 UTC from IEEE Xplore. Restrictions apply.

4. reliability (rb): the degree of which a microservice to perform the required functionality responses properly (unit: number of failures per 5 day period/total requests)
5. cost (ct): the cost per microservice request or invocation (unit: cents per service request)

mQoS parameters can possess an increasing or decreasing characteristics. An mQoS parameter has an increasing magnitude if higher mQoS values correspond to better mQoS and vice versa. An mQoS parameter has a decreasing magnitude if lower mQoS values correspond to better mQoS. For instance, throughput is considered an increasing parameter due to the fact that a microservice with high throughput is considered to be better than one with a low throughput. On the other hand, response time is considered a decreasing mQoS parameter since reducing response time translates to faster performance and/or processing.

B. *Monitoring Microservices*

Since microservices reply on resources such as network connectivity and server or cloud capabilities, their mQoS may fluctuate during their lifecycle. Therefore, providing mQoS metric for a microservice at a specific point in time may not be that accurate due to the differences in mQoS measurements that may arise in the microservice's performance over time. Hence, we included as part of our mQoSM framework the mQoSOP and mQoSS to monitor microservices ad evaluate mQoS metrics periodically.

Monitoring the behavior of microservices as part of our mQoSM framework offers many advantages including: (a) it provides non-functional properties of microservices to be used as part of the discovery process; (b) allows clients or IoT applications to invoke a microservice with confidence; (c) enables a way to rank microservices over time; (d) facilitates guarantee of measurements of service qualities; and (e) offers flexibility and extensibility for measuring the accuracy of mQoS information (e.g. extend to data distribution service).

As more and more microservice implementations become available or integrated into IoT applications, multiple microservices may compete in their service offerings. Although multiple microservices may offer the same functionality, they vary in the degree to which these functionalities are delivered with respect to mQoS. To be more specific, mQoS is a significant differentiating factor that may drive the popularity, stability and overall worthiness of a microservice.

To measure mQoS parameters, we utilize a middleware approach that can assess microservices in performing the intended functionality. This middleware can be integrated locally within an IoT ecosystem or hosted on a cloud-based platform. The choice of deploying mQoSM as a middleware approach offers many advantages such as the ability to control the way mQoS measurements are executed, continuously monitor behavior of microservices and store information in a repository which can be used to perform further analysis on microservices.

V. Experiments and Results

Data used in this paper are based on actual microservice implementations that are available over the web and their functionality meets that of the requirements of CPI's operating scenario introduced earlier. This data represents microservices that are external to the CPI's IIoT system (i.e. over the cloud). These microservices were listed in Mashape [16], ProgrammableWeb [20], Mashery [21] and CloudRail [22]. Five microservices were chosen from the same domain and they all share the same functionality of sending SMS messages (with pictures) as part of a notifications layer for CPI's maintenance order generation from the operating scenario we introduced earlier. mQoS parameters discussed in Section IV were used to evaluate mQoS metrics for all of the five microservices. The mQoS cost is represented in cents and was provided by the service provider. In addition, availability values were measured over a five-day period.

TABLE I. mQoS Metrics for Various Collected Microservices

ID	Service Provider	rt (ms)	tp (req/min)	av (%)	rb (%)	ct (cents US)
1	SMS Fusion (smsfusion.com.au)	1957	8.5	96	85	6
2	Twilio SMS API (twilio.com)	960	16	98	97	20
3	Apifonica SMS API (apifonica.com)	1420	9	88	74	12
4	SMSAPI (smsapi.com)	820	12	91	87	5
5	ClickSend SMS (clicksend.com)	1060	7	86	89	10
	average	1244	10.5	92	86	10.6

Table 1 shows mQoS values that were measured by mQoSM for five microservices. In an attempt to find whether these microservices are relevant to CPI, there needs to exist a set of microservices that share the same functionality and are used to measure mQoS metrics based on a QoS criteria. For CPI software engineers to arbitrarily choose an SMS API from the list shown in Table 1, they risk that performance degradation in terms of the end-to-end quality of CPI's operating efficiency.

In the case CPI software engineers consider cost as the driving factor for choosing the best microservice, based on Table 1, the microservice SMSAPI with id four would be selected for integration. However, this microservice's overall availability is slightly less than the average and in cases of mission critical systems, we would risk that the microservice is not available when it is most needed. Hence, considering one mQoS factor is not ideal. When considering possible microservices into an IIoT application, it is important to take into consideration the various mQoS measurements.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:43:35 UTC from IEEE Xplore. Restrictions apply.

Considering mQoS measurements at a specific point in time may not be ideal. That is, it is important to consider historical values of these mQoS measurements over a period of time. The results in Table 1 represent averages over a period of five days for each microservice. Ideally, the more data that can be collected over longer duration, the more accurate are the indicators of how accurate these measured values are. We believe that by integrating mQoSM into IIoT applications such as the of CPI can be used in the long term (e.g. 6 months or more) to collect sufficient data that can be used by software engineers or business analysts to make accurate decisions with respect to the deployment of proper microservices that meet IIoT application demands. Therefore, a cost optimization function or a ranking procedure that can provide relative ranking of microservices based on their mQoS becomes inevitable particularly as the number of microservices in IoT applications begins to increase.

VI. Conclusion

In this paper, we presented the microservices Quality of Service (mQoSM) management framework. mQoSM is a QoS-aware middleware that measures the overall quality of microservices for possible integration in IIoT applications. The use of non-functional properties for microservices significantly improves the probability of having stable and durable microservices architecture. The proposed solution has shown usefulness and effectiveness of incorporating mQoS parameters as part of the selection criteria and in differentiating between microservices that share similar functionalities. Discriminating on the selection of appropriate microservices relies on the client's ability to determine relevant mQoS parameters. In order to allow for different mQoS circumstances, there is an apparent need to weight each quality factor relative to the importance or magnitude that it endows upon ranking microservices based on mQoS parameters. For future work, we plan to extend our mQoSM middleware to include a ranking mechanism for enabling applications (or clients) to determine relevant microservices in addition to tracking the behavior of microservices over time.

References

[1] Data distribution services specification, V1.2, Object Management Group (OMG), Needham, MA, USA, Apr. 2, 2015. www.omg.org/spec/DDS/1.2/

[2] Real-Time Innovations Inc. (RTI), "Built-in QoS profiles," 2017. [Online]. Available: http://blogs.rti.com/2014/02/11/built-in-qos-profiles

[3] A. Agirre, J. Parra, E. Estévez and M. Marcos, "QoS aware platform for dependable sensory environments," 2014 IEEE International Conference on Multimedia and Expo Workshops (ICMEW), Chengdu, 2014, pp. 1-5.

[4] J. F. Inglés-Romero, A. Romero-Garcés, C. Vicente-Chicote and J. Martínez, "A Model-Driven Approach to Enable Adaptive QoS in DDS-Based Middleware," in IEEE Transactions on Emerging Topics in Computational Intelligence, vol. 1, no. 3, pp. 176-187, June 2017.

[5] Vortex OpenSplice Modeler, 2017. [Online]. Available: http://www.sprismtech.com/vortex/vortex-opensplice/tools/modeler

[6] J. Gray et al., "Concern separation for adaptive QoS modeling in distributed real-time embedded systems," in Behavioral Modeling for Embedded Systems and Technologies: Applications for Design and Implementation, Information Science Reference. Hershey, PA, USA: IGI Global, 2009.

[7] D. S. Joseph, W. Hoffert, and A. Gokhale, "DQML: A modeling language for configuring distributed publish/subscribe quality of service policies," in Proc. 10th Int. Symp. Distrib. Objects, Middleware, Appl., 2008, pp. 515–534

[8] P. Boonma and J. Suzuki, "Self-configurable publish/subscribe middleware for wireless sensor networks," in Proc. 6th IEEE Conf. Consum. Commun. Netw., 2009, pp. 1376–1383.

[9] X. Wang, Y. Chen, C. Lu, and X. Koutsoukos, "FC-ORB: A robust distributed real-time embedded middleware with end-to-end utilization control," J. Syst. Softw., vol. 80, no. 7, pp. 938–950, 2007.

[10] S. Ran, "A Model for Web Services Discovery with QoS," ACM SIGecom Exchanges 4(1). 1-10, 2003.

[11] A. Sheth, J. Cardoso, J. Miller, K. Kochut, "QoS for Service-oriented Middleware," Proceedings of the 6th World Multiconference on Systemics, Cybernetics and Informatics Volume 8. 528-534, 2002.

[12] Y. Liu, A. NGU, L. Zeng, "QoS computation and policing in dynamic web service selection," World Wide Web Conference. 66-73, 2004.

[13] V. Tosic, B. Pagurek, K. Patel, B. Esfandiari, W. Ma, "Management Applications of the Web Service Offerings Language (WSOL)," 15th International Conference on Advanced Information Systems Engineering Austria, 2003.

[14] L. Jin, V. Machiraju, A. Sahai, "Analysis on Service Level Agreement of Web Services. Software Technology Laboratory, HP Laboratories, HPL, 2002.

[15] P. Štefanic, M. Cigale, A. Jones and V. Stankovski, "Quality of Service Models for Microservices and Their Integration into the SWITCH IDE," 2017 IEEE 2nd International Workshops on Foundations and Applications of Self* Systems (FAS*W), Tucson, AZ, 2017, pp. 215-218.

[16] Mashape – Free API Management Platform and Marketplace, https://market.mashape.com/explore, Last Accessed June 7, 2018).

[17] E. Al-Masri, and Q.H. Mahmoud, "Toward Quality-Driven Web Service Discovery," IEEE IT Professional, 10(3), pp. 24-28, May/Jun, 2008.

[18] E. Al-Masri, and Q.H. Mahmoud, "Investigating Web Services on the World Wide Web". In Proceedings of the 17th International World Wide Web Conference (WWW2008), pp. 795-804, 2008.

[19] International Telecommunication Union, "ITU-T Recommendation E.800: Terms and Definitions Related to Quality of Service and Network Performance Including Dependability," International Telecommunication Union -ITU, 1994.

[20] ProgrammableWeb APIs, Mashups and the Web as Platform, https://www.programmableweb.com, Last Accessed June 7, 2018).

[21] Tibco Mashery API Management Platform, https://www.mashery.com, Last Accessed June 7, 2018).

[22] CloudRail API Integration Solution, https://cloudrail.com, Last Accessed June 7, 2018).

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 21:43:35 UTC from IEEE Xplore. Restrictions apply.